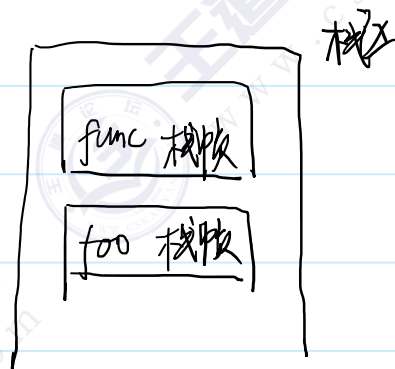


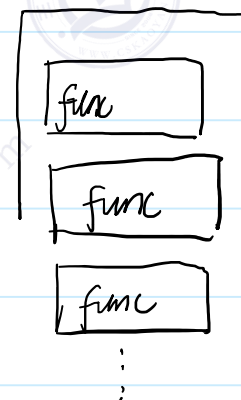
递归

设计一个函数func的函数定义，在func的函数体里面，去调用func函数本身 ---> 递归

```
void func(){  
    foo();  
}
```



```
void func(){  
    func();  
}
```



无限递归带来的问题

```
void func(int i) {  
    printf("i = %d\n", i);  
    func(i - 1);  
}  
  
int main() {  
    func(4);  
    return 0;  
}
```



由于栈区里面的栈帧太多，导致了栈溢出问题。

合理地设置递归出口

```
void func(int i) {  
    printf("i = %d\n", i);  
    // 避免无限递归触发栈溢出，我们可以设计一个递归出口  
    if (i <= 0) {  
        return;  
    }  
    else {  
        func(i - 1);  
    }  
}  
int main() {  
    func(4);  
    return 0;  
}
```

爬楼梯问题

假设你正在爬一个楼梯。它总共有 n 阶台阶。

每次你只能爬 1 阶 或者 爬 2 阶。

问：你有多少种不同的方法可以爬到楼顶？

Handwritten list of climbing sequences for $n=1$ to $n=5$:

$num = 1$	1	$num = 4$	1→1→1→1
$num = 2$	1→1		1→1→2
	2		1→2→1
$num = 3$	1→1→1		2→1→1
	1→2		2→2
	2→1		
	$num = 5$		
	⋮		
	n		

我们从传统的全局视角当中，很难去找到问题的解决方案

局部解决：把一个规模大一点的问题转移成规模小一点的问题

分治法

递推

递归

分而治之 divide and conquer

假设 $num=1, 2, 3, 4, \dots, n-1$ 都已经解决了, 在前面的基础上, 去解决 $num = n$ 的问题

组合计数问题-加法原理

num 为 n 的数量 = num 为 $n-1$ 的数量 + num 为 $n-2$ 的数量

大问题可以转移成小问题

$n \rightarrow n-1 \quad n-2$
 $n-1 \rightarrow n-2 \quad n-3$
 $n-2 \rightarrow n-3 \quad n-4$

...

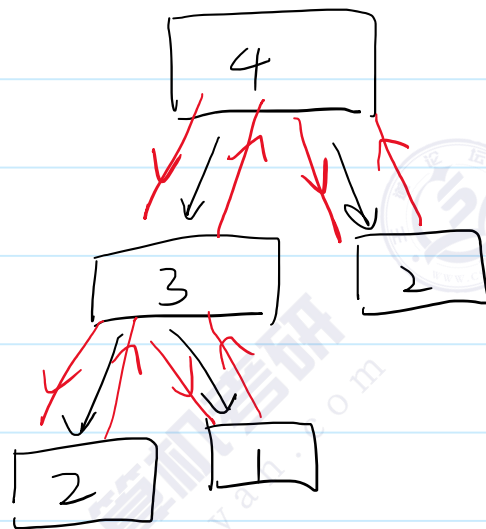
3 $\rightarrow$ 2 1

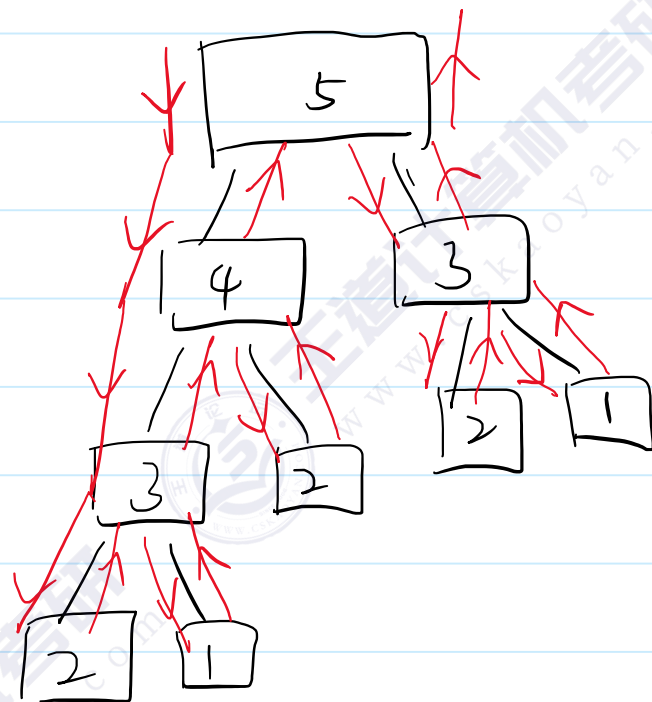
```

int f(int n) {
    if (n > 2) {
        // 大问题 --> 小问题 --> 递归
        int result = f(n - 1) + f(n - 2);
        return result;
    }
    // 最小问题的解决方案
    else if (n == 2) {
        return 2;
    }
    else if (n == 1) {
        return 1;
    }
}

int main() {
    int n = 4;
    printf("f(n) = %d\n", f(n));
    return 0;
}

```





递归

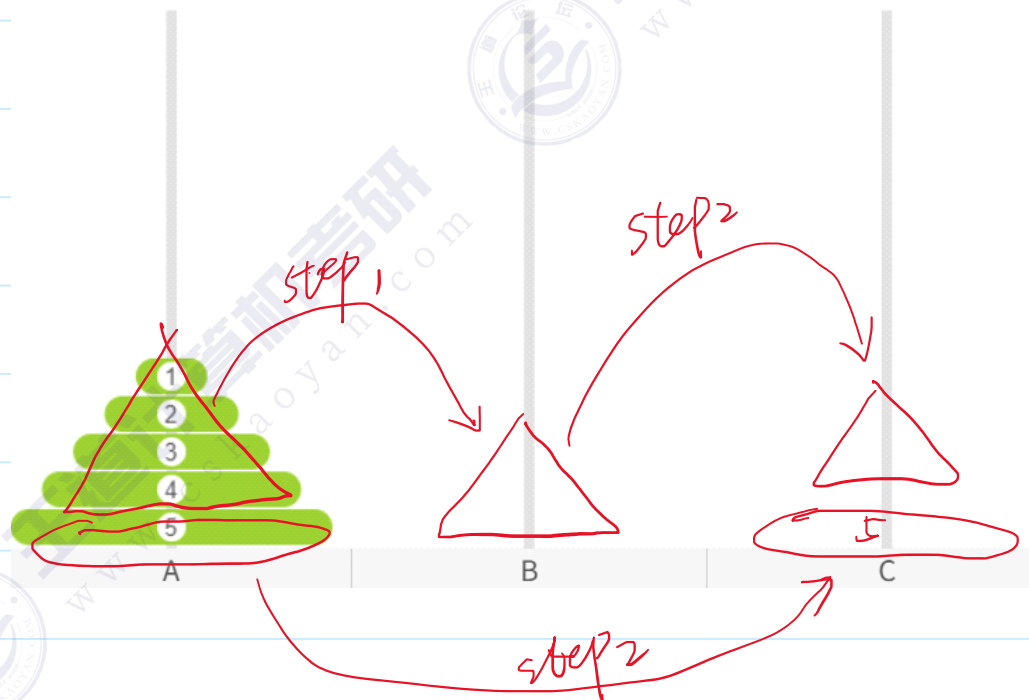
大问题-->小问题 递推

问题的边界 回归

汉诺塔问题

现在有 n 个从小到大排列个圆盘，我们需要将圆盘全部移动到另一个柱子上，并遵守以下规则：

1. 每次只能移动一个圆盘
2. 移动过程中，大盘不能放在小盘上面
3. 只能移动最顶端的圆盘



汉诺塔问题的分而治之方案

- 1、将上面的前 $n-1$ 个盘子 从 A移动到B
- 2、将最底下的第 n 个盘子 从 A移动到C
- 3、再将B上面的前 $n-1$ 个盘子 从B移动到C

如果 n 为1, 那么直接移动即可

汉诺塔代码

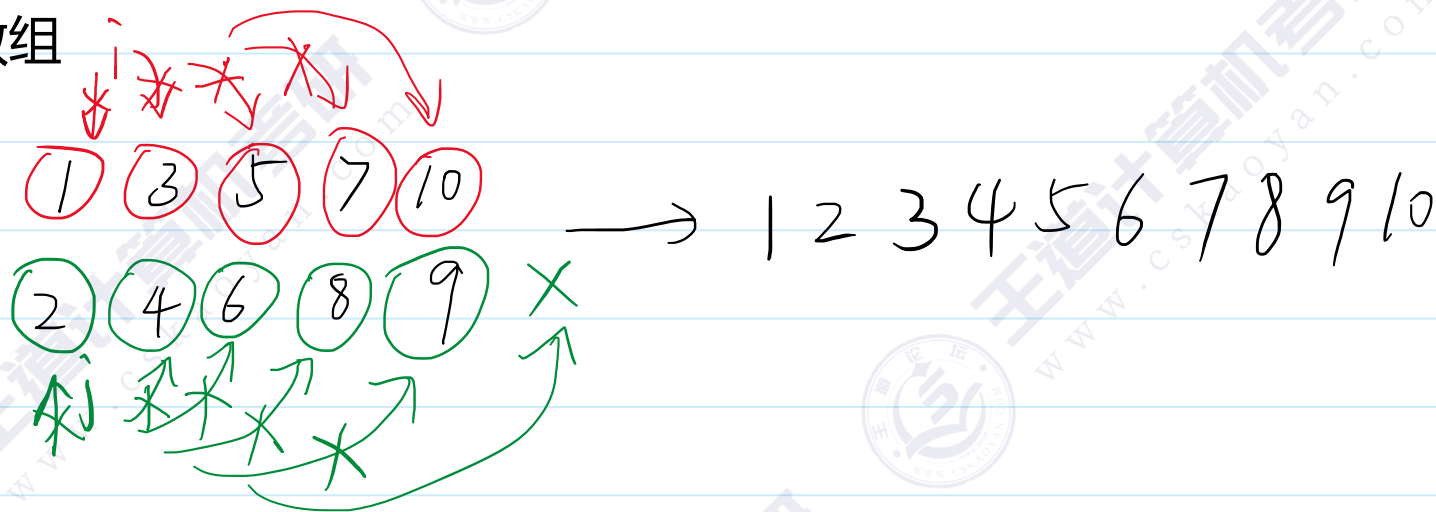
```
void hanoi(int n, char from, char buffer, char to) {  
    // from 圆盘的原来位置  
    // to 圆盘的最终位置  
    // buffer 临时借用的位置  
    if (n > 1) {  
        // 前n-1个圆盘从 from 到 buffer  
        hanoi(n - 1, from, to, buffer);  
        hanoi(1, from, buffer, to);  
        // 前n-1个圆盘从 buffer到to  
        hanoi(n - 1, buffer, from, to);  
    }  
    else if (n == 1) {  
        printf("move %c to %c\n", from, to);  
    }  
}  
  
int main() {  
    int n = 5;  
    hanoi(n, 'A', 'B', 'C');  
    return 0;  
}
```

```
move A to C  
move A to B  
move C to B  
move A to C  
move B to A  
move B to C  
move A to C  
move A to B  
move C to B  
move C to A  
move B to A  
move C to B  
move A to C  
move A to B  
move C to B  
move A to C  
move B to A  
move B to C  
move A to C  
move B to A  
move C to B  
move C to A  
move B to A  
move B to C  
move A to C  
move A to B  
move C to B  
move A to C  
move B to A  
move B to C  
move A to C
```

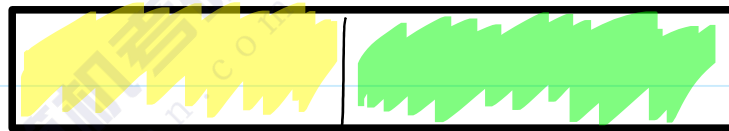
利用分而治之的思想进行排序

现有一个无序的数组 ($i < j$ 不一定有 $a[i] < a[j]$)

给出两个有序的小数组，如何可以合并成一个大的有序数组



1、将大问题转移成小问题



2、最小问题的解决方案



当数组有 0 or 1 个元素时，
数组自然有序。

有序

有序

合并有序数组

合并的功能

给出一个数组arr left,mid 有序 mid+1,right 有序

准备一个临时数组temp



```
void merge(int* arr, int* temp, int left, int mid, int right) {  
    // arr[left] ~ arr[mid] 和 arr[mid+1] ~ arr[right] 进行合并  
    // 1 将arr的内容备份到temp里面  
    for (int i = left; i <= right; ++i) {  
        temp[i] = arr[i];  
    }  
    // i访问左半边 j访问右半边 k存放结果  
    int i, j, k;  
    for (i = left, j = mid + 1, k = left; i <= mid && j <= right; ++k) {  
        if (temp[i] < temp[j]) {  
            arr[k] = temp[i];  
            ++i;  
        }  
        else {  
            arr[k] = temp[j];  
            ++j;  
        }  
    }  
    while (i <= mid) {  
        arr[k] = temp[i];  
        ++i;  
        ++k;  
    }  
    while (j <= right) {  
        arr[k] = temp[j];  
        ++j;  
        ++k;  
    }  
}
```

//归并排序

```
void mergeSort(int *arr, int *temp, int left, int right) {  
    if (left < right) {  
        int mid = (left + right) / 2;  
        mergeSort(arr, temp, left, mid);  
        mergeSort(arr, temp, mid + 1, right);  
        merge(arr, temp, left, mid, right);  
    }  
}  
  
int main() {  
    int arr[] = { 3,87,2,93,78,56,61,38,12,40 };  
    int temp[10];  
    mergeSort(arr, temp, 0, 9);  
    return 0;  
}
```